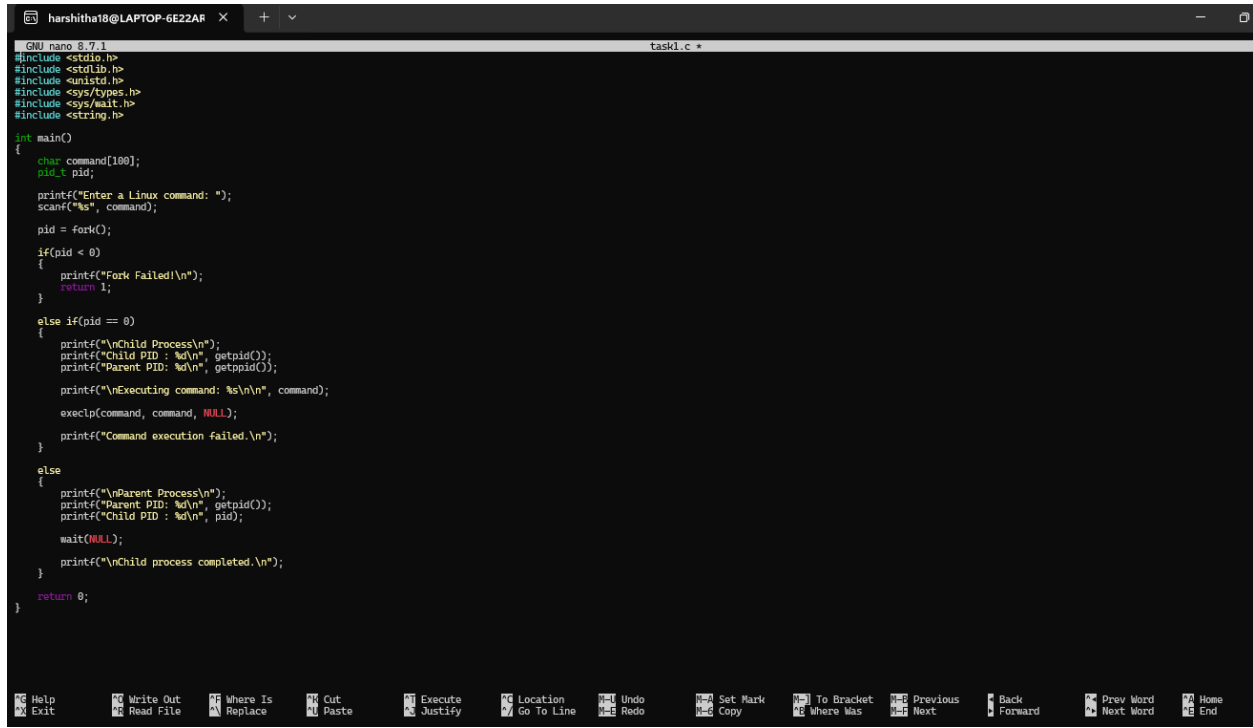


WEEK-2:

TASK-1;

```
harshitha18@LAPTOP-6E22AR6C:~$ nano task1.c
```



```
harshitha18@LAPTOP-6E22AR6C:~$ nano task1.c
GNU nano 8.7.1 task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if(pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }
    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: %s\n", command);
        execlp(command, command, NULL);

        printf("Command execution failed.\n");
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

The screenshot shows a nano text editor window with the title bar "harshitha18@LAPTOP-6E22AR6C:~\$ nano task1.c". The editor contains a C program that prompts the user for a Linux command, forks a child process to execute it, and prints various process IDs and status messages. The code is as follows:

```
GNU nano 8.7.1 task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if(pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }
    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: %s\n", command);
        execlp(command, command, NULL);

        printf("Command execution failed.\n");
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

The bottom of the window features a toolbar with various editing and navigation icons and their corresponding keyboard shortcuts.

```
harshitha18@LAPTOP-6E22AR  X  +  v
harshitha18@LAPTOP-6E22AR6C:~$ nano task1.c
harshitha18@LAPTOP-6E22AR6C:~$ gcc task1.c -o task1
harshitha18@LAPTOP-6E22AR6C:~$ ./task1
Enter a Linux command: ls

Parent Process
Parent PID: 2206
Child PID : 2207

Child Process
Child PID : 2207
Parent PID: 2206

Executing command: ls

OSSP      file1.text  fork_example.c  task1    task5
backup.txt file1.txt      hello           task1.c  task5.c
exec_example file2.txt    hello.c         task4
exec_example.c fork_example process.c       task4.c

Child process completed.
harshitha18@LAPTOP-6E22AR6C:~$ pwd
/home/harshitha18
harshitha18@LAPTOP-6E22AR6C:~$ ./task1
Enter a Linux command: pwd

Parent Process
Parent PID: 2229
Child PID : 2230

Child Process
Child PID : 2230
Parent PID: 2229

Executing command: pwd

/home/harshitha18

Child process completed.
harshitha18@LAPTOP-6E22AR6C:~$ |
```

This program demonstrates process creation using `fork()`, command execution using `exec1p()`, and process synchronization using `wait()`. It also shows how parent and child processes execute independently while sharing a parent-child relationship.

Task-2;

```
harshitha18@LAPTOP-6E22AR  X  +  v
harshitha18@LAPTOP-6E22AR6C:~$ uname -a
Linux LAPTOP-6E22AR6C 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
harshitha18@LAPTOP-6E22AR6C:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               39 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      2
On-line CPU(s) list:        0,1
Vendor ID:                   GenuineIntel
Model name:                  Intel(R) Celeron(R) N4500 @ 1.10GHz
CPU family:                  6
Model:                      156
Thread(s) per core:         1
Core(s) per socket:         2
Socket(s):                   1
Stepping:                    0
BogoMIPS:                    2227.20
Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep
                             mtrr pge mca cmov pat pse36 clflush mmx fx
                             sr sse sse2 ss ht syscall nx rdtscp lm cons
                             tant_tsc arch_perfmon rep_good nopl xtopolo
                             gy tsc_reliable nonstop_tsc cpuid tsc_known
                             _freq pni pclmulqdq vmx ssse3 cx16 pdcm sse
                             4_1 sse4_2 x2apic movbe popcnt tsc_deadline
                             _timer aes xsave rdrand hypervisor lahf_lm
                             3dnowprefetch ssbd ibrs ibpb stibp ibrs_enh
                             anced tpr_shadow ept vpid ept_ad fsgsbase t
                             sc_adjust smep erms rdseed smap clflushopt
                             clwb sha_ni xsaveopt xsavec xgetbv1 xsaves
                             vnm i umip waitpkg rdpid movdiri movdir64b m
                             d_clear flush_lld arch_capabilities

Virtualization features:
Virtualization:              VT-x
Hypervisor vendor:           Microsoft
Virtualization type:         full
Caches (sum of all):
```

```
harshitha18@LAPTOP-6E22AR  X  +  v
L1d: 64 KiB (2 instances)
L1i: 64 KiB (2 instances)
L2: 3 MiB (2 instances)
L3: 4 MiB (1 instance)
NUMA:
  NUMA node(s): 1
  NUMA node0 CPU(s): 0,1
Vulnerabilities:
  Gather data sampling: Not affected
  Ghostwrite: Not affected
  Indirect target selection: Mitigation; Aligned branch/return thunks
  Itlb multihit: Not affected
  L1tf: Not affected
  Mds: Not affected
  Meltdown: Not affected
  Mmio stale data: Mitigation; Clear CPU buffers; SMT Host state unknown
  Old microcode: Not affected
  Reg file data sampling: Mitigation; Clear Register File
  Retbleed: Mitigation; Enhanced IBRS
  Spec rstack overflow: Not affected
  Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS Not affected; BHI SW loop, KVM SW loop
  Srbds: Unknown: Dependent on hypervisor status
  Tsa: Not affected
  Tsx async abort: Not affected
  Vmscape: Not affected
harshitha18@LAPTOP-6E22AR6C:~$ ps
  PID TTY          TIME CMD
  2243 pts/0    00:00:00 bash
  2308 pts/0    00:00:00 ps
harshitha18@LAPTOP-6E22AR6C:~$ top
```

```
harshitha18@LAPTOP-6E22AR x + v
harshitha18@LAPTOP-6E22AR6C:~$ top
top - 16:31:22 up 2:23, 1 user, load average: 0.02, 0.01, 0.00
Tasks: 25 total, 1 running, 24 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.2 sy, 0.0 ni, 99.8 id, 0.0 wa, 0.0 hi, 0.0 si,
MiB Mem : 17891.0 total, 17328.2 free, 569.5 used, 209.5 buff/c
MiB Swap: 5120.0 total, 5120.0 free, 0.0 used. 17321.6 avail

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+
    1 root        20   0   24568   15752   11620 S   0.0   0.1   0:02.29
    2 root        20   0    3180    2096    1964 S   0.0   0.0   0:00.03
    7 root        20   0    3180    2048    1940 S   0.0   0.0   0:00.00
   57 root        19  -1   50400   16960   15640 S   0.0   0.1   0:00.46
   91 systemd+    20   0   22416   14508   12016 S   0.0   0.1   0:00.11
  102 root        20   0   35436   12448    9256 S   0.0   0.1   0:02.74
  123 root        20   0    2888    2008    1884 S   0.0   0.0   0:00.04
  124 root        20   0    4472    3152    2908 S   0.0   0.0   0:00.02
  125 message+   20   0    8820    5568    4828 S   0.0   0.0   0:00.15
  129 root        20   0   44096   29856   14716 S   0.0   0.2   0:00.49
  153 root        20   0   18424    9320    8088 S   0.0   0.1   0:00.15
  174 syslog      20   0  220548    5680    4724 S   0.0   0.0   0:00.10
  200 _chrony      20   0   20424    6248    5476 S   0.0   0.0   0:00.13
  208 _chrony      20   0   12096    2492    1852 S   0.0   0.0   0:00.00
  215 root        20   0    5492    2812    2624 S   0.0   0.0   0:00.01
  216 root        20   0  123460   32652   18040 S   0.0   0.2   0:00.46
  320 root        20   0    8644    5584    4744 S   0.0   0.0   0:00.03
  385 harshit+   20   0   22752   13428   10968 S   0.0   0.1   0:00.26
  389 harshit+   20   0   22912    4080    2080 S   0.0   0.0   0:00.00
  418 harshit+   20   0    6136    5476    3928 S   0.0   0.0   0:00.05
  693 polkitd     20   0  306444    8520    7704 S   0.0   0.0   0:00.29
 2239 root        20   0    3188    1116     980 S   0.0   0.0   0:00.00
 2241 root        20   0    3204    1256    1108 S   0.0   0.0   0:00.01
 2243 harshit+   20   0    6268    5608    3936 S   0.0   0.0   0:00.11
 2318 harshit+   20   0    8200    5964    3892 R   0.0   0.0   0:00.01

harshitha18@LAPTOP-6E22AR6C:~$ |
```

The experiment demonstrated how Linux abstracts hardware resources by providing system services that efficiently manage CPU, memory, storage, and I/O devices. The terminal commands helped analyze system information, processor details, and running processes in real time.